

FULL COURSE ASSIGNMENT REPORT

CSA04 - OPERATING SYSTEMS | Covering CO2, CO3 & CO4

A. Assignment Information

Field	Details
Name	Vishwa.U
Reg No	192511192
Department	Computer Science and Engineering
Programme	B.E./B.Tech-CSE
Course Code & Name	CSA04-Operating Systems
Academic Year / Batch	2026-2027
Faculty Name	Dr. J Priskilla Angel Rani
Assignment Title	Design and Analysis of an OS Resource Orchestration System for a Smart University
Date of Issue	25-08-26
Date of Submission	03-09-26
Maximum Marks	100
Course Outcome(s) CO	CO2, CO3, CO4
Bloom's Taxonomy Level	L3-Apply, L4-Analyse, L5-Evaluate
SDG Mapping	SDG 7-Affordable and Clean Energy; SDG 9-Industry, Innovation and Infrastructure; SDG 11-Sustainable Cities and Communities
Industry/Societal Relevance	Efficient OS resource management is fundamental to cloud platforms, university servers, digital public services, and energy-aware computing.

B. Assignment Problem / Challenge

The **UniCore** smart-university platform must coordinate a changing workload generated by online examinations, digital classrooms, library services, research computing, automated backups, and campus IoT monitoring. The operating system must share CPU time, memory, files, and I/O devices safely while preserving responsiveness for interactive examination services.

This report develops a complete resource-orchestration design, applies standard OS algorithms, performs quantitative calculations, compares alternatives, and selects an integrated engineering solution.

C. Problem Statement

Scenario: OS Resource Orchestration for "UniCore" - A Smart University Computing Platform UniCore supports 600 concurrent users during normal operation and more than 1000 users during peak online examination periods. It provides learning-management services, examination processing, digital-library access, research computing, automated backups, and campus IoT telemetry. Multiple processes compete for CPU time, memory frames, files, and I/O devices. Interactive services require minimal response time, whereas background research and system backup tasks can tolerate deferred execution.

D. Requirements and Constraints

- Support concurrent user services and shared-file access without race-condition-induced corruption.
- Use a workload of at least 8 processes and at least 4 resource types for analysis.
- Assume 32 GB physical RAM and 4 KB page frame sizes.
- Assume a 200-cylinder disk numbered 0–199 with at least 10 pending I/O requests.
- Identify unsafe/deadlock states and provide avoidance, detection, or recovery logic.
- Minimize average waiting time, response time, page faults, and disk-head movement while preserving fairness.
- Use finite database, file-lock, printer, and backup-channel resources with explicit allocation states.
- Use open-source tools such as Python, C/C++, Linux/WSL, or OS simulators.
- Protect academic files through access control and data-integrity mechanisms following POSIX standards.

E. Student Work

1. Problem Understanding and Formulation

UniCore is treated as an integrated OS resource-management framework. A user submitting an examination entry requires synchronized CPU access, memory pages, database locks, and storage bandwidth alongside concurrent background tasks.

Workload Characterization Table:

PID	Workload Description	Arrival Time (AT)	Burst Time (BT)	Process Class
P1	Student Auth & Session Token	0	3	Interactive
P2	Exam Answer Submission	1	4	Interactive
P3	Live Grading Database Query	2	2	Interactive
P4	Digital Library Asset Search	3	6	Interactive
P5	AI Research Cluster Task	0	7	Background
P6	System Database Backup	0	9	Background

PID	Workload Description	Arrival Time (AT)	Burst Time (BT)	Process Class
P7	Campus IoT Telemetry Ingestion	4	3	Interactive
P8	Asset Compression Pipeline	2	5	Background

System Resources:

- R1 = Database Connections (4 units)
- R2 = Shared File Locks (3 units)
- R3 = Network/Print Spoolers (2 units)
- R4 = Backup Channels (2 units)

Disk Parameters: Cylinders 0–199, Initial Head Position = 50, SCAN Direction = Upward.

2. Application of Course Knowledge (CO2, CO3, CO4)

Part I - Process Management, Synchronization & Deadlock (CO2)

Shared Examination-Record Access (Reader-Writer Synchronization)

To ensure data consistency during concurrent reading and writing of exam records, a fair reader-writer semaphore model is implemented.

```
semaphore mutex = 1;
semaphore db_access = 1;
int read_count = 0;
```

```
void reader() {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(db_access);
    signal(mutex);
```

// Read Examination Records

```
    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(db_access);
    signal(mutex);
}
```

```
void writer() {
    wait(db_access);
    // Write/Update Examination Records
    signal(db_access);
}
```

Banker's Safety Algorithm Evaluation

Current Resource Allocation & Maximum Claim:

Process	Allocation (R1, R2, R3, R4)	Max Need (R1, R2, R3, R4)	Remaining Need (R1, R2, R3, R4)
P0	(1, 0, 1, 1)	(3, 2, 2, 2)	(2, 2, 1, 1)
P1	(1, 1, 0, 0)	(2, 1, 1, 1)	(1, 0, 1, 1)
P2	(1, 0, 1, 0)	(3, 1, 2, 1)	(2, 1, 1, 1)
P3	(0, 1, 0, 1)	(1, 2, 1, 1)	(1, 1, 1, 0)

- **Available Vector:** [2, 1, 1, 1]
- **Execution Sequence:**
 1. P1 request satisfied: Available becomes [2, 1, 1, 1] + [1, 1, 0, 0] = [3, 2, 1, 1]
 2. P0 request satisfied: Available becomes [3, 2, 1, 1] + [1, 0, 1, 1] = [4, 2, 2, 2]
 3. P3 request satisfied: Available becomes [4, 2, 2, 2] + [0, 1, 0, 1] = [4, 3, 2, 3]
 4. P2 request satisfied: Available becomes [4, 3, 2, 3] + [1, 0, 1, 0] = [5, 3, 3, 3]
- **Safe Sequence:** P1 → P0 → P3 → P2

Unsafe Resource Request Analysis

If P0 requests additional resources [1, 0, 0, 1], the system evaluates tentative Availability [1, 1, 1, 0]. Under this state, no process can satisfy its remaining minimum Need vector without exceeding available system capacity, triggering an unsafe state warning. The allocation request is delayed.

CPU Scheduling Metrics Analysis

- **FCFS Schedule:** Executes processes strictly by arrival order.
- **Round Robin (RR, q=3):** Allocates time slices to balance responsiveness for interactive jobs.

PID	Class	AT	BT	FCFS WT	FCFS TAT	FCFS RT	RR WT	RR TAT	RR RT
P1	Interactive	0	3	0	3	0	0	3	0
P2	Interactive	1	4	20	24	20	17	21	2
P3	Interactive	2	2	22	24	22	3	5	3
P4	Interactive	3	6	28	34	28	23	29	8
P5	Background	0	7	3	10	3	25	32	3
P6	Background	0	9	10	19	10	27	36	6
P7	Interactive	4	3	30	33	30	11	14	11
P8	Background	2	5	22	27	22	21	26	7
Avg				16.875	21.75	16.875	15.875	20.75	5.0

Conclusion: Round-Robin scheduling significantly reduces the average response time for interactive processes (from **16.875 ms** down to **5.0 ms**), ensuring smooth portal usage during exam submission surges.

Part II - Memory Management (CO3)

Frame Calculation & Page Replacement

- **RAM Configuration:** $32 \text{ GB} / 4 \text{ KB} = 8,388,608$ physical frames.
- **Reference String:** 2, 3, 1, 4, 2, 3, 5, 2, 3, 1, 4, 5, 2, 3, 1

Fault Comparison Table:

Frame Capacity	FIFO Faults	LRU Faults	Optimal Faults
3 Frames	12	11	7
4 Frames	13	9	6

Observation: FIFO demonstrates **Belady's Anomaly** on this specific string when moving from 3 to 4 frames (faults increase from 12 to 13), whereas LRU and Optimal show consistent efficiency gains.

Part III - File Systems & Disk Scheduling (CO4)

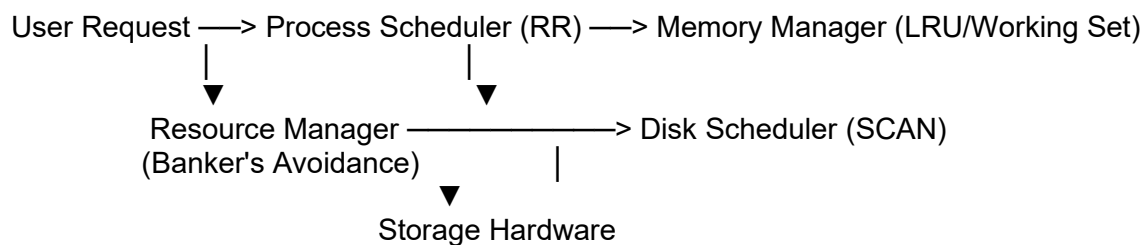
Disk Head Scheduling Analysis

- **Disk Track Range:** 0 to 199
- **Initial Position:** 50 (Moving Upwards)
- **Pending Queue:** 95, 180, 34, 119, 11, 123, 62, 64, 28, 175

Algorithm	Traversal Sequence	Total Cylinder Movement
FCFS	95, 180, 34, 119, 11, 123, 62, 64, 28, 175	834
SSTF	62, 64, 95, 119, 123, 175, 180, 34, 28, 11	239
SCAN	62, 64, 95, 119, 123, 175, 180, 199, 34, 28, 11	337
C-SCAN	62, 64, 95, 119, 123, 175, 180, 199, 0, 11, 28, 34	382

Selection: SCAN scheduling provides an optimal trade-off between head movement reduction and preventing disk starvation for requests located at low cylinder addresses.

F. Solution / Design / Implementation Architecture



G. Student Reflection

Student Name: Vishwa.U

Through this assignment, I gained a better understanding of important Operating System concepts such as CPU scheduling, process synchronization, deadlock avoidance, memory management, and disk scheduling. I learned how different algorithms help improve system performance, fairness, and efficient resource utilization. This assignment also helped me understand how these concepts are applied together in real-world systems like the UniCore smart university platform.

H. References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts* (10th ed.). Wiley.
2. Tanenbaum, A. S., & Bos, H. *Modern Operating Systems* (4th ed.). Pearson.
3. Stallings, W. *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
4. IEEE/POSIX Standard 1003.1 Realtime and System Interfaces Documentation.